

O Problema do Clique e sua Aplicação em Análise de Propagação Viral

Paulo E. R. Araujo¹, Guilherme A. Avelino²

¹Universidade Federal do Piauí – UFPI
Teresina – PI – Brasil

pumbadeveloper@gmail.com

Abstract. *This article presents the maximum clique problem in graphs, its relevance as an NP-complete problem, and a practical application to selecting influencer groups for social-proof-driven marketing campaigns. Candidate influencers are modeled as content creators whose audiences overlap. When a group of creators who all share audiences pairwise endorses the same product, the common audience receives that endorsement from several trusted sources at once, which triggers social proof and the bandwagon effect and raises adoption. We implement the case study in TypeScript using deterministic and reproducible instances. For each category we build a graph and find its maximum clique with an exact algorithm and with a greedy heuristic, and we present the results in an interactive HTML report. We also compare the two algorithms across input sizes to make explicit the time–quality trade-off of exchanging optimality for polynomial-time scalability.*

Resumo. *Este artigo apresenta o problema do clique máximo em grafos, sua relevância como problema NP-completo e uma aplicação prática à seleção de grupos de influenciadores para campanhas de marketing baseadas em prova social. Os candidatos são modelados como criadores de conteúdo cujas audiências se sobrepõem. Quando um grupo de criadores que compartilham audiência par a par endossa o mesmo produto, o público comum recebe esse endosso de várias fontes confiáveis ao mesmo tempo, o que ativa a prova social e o efeito manada e eleva a adesão. Implementa-se o estudo de caso em TypeScript com instâncias determinísticas e reproduzíveis. Para cada categoria constrói-se um grafo e busca-se o clique máximo com um algoritmo exato e com uma heurística gulosa, e os resultados são apresentados em um relatório HTML interativo. Comparamos ainda os dois algoritmos em diferentes tamanhos de entrada para explicitar o compromisso tempo–qualidade de trocar otimalidade por escalabilidade em tempo polinomial.*

1. Introdução

O problema do clique em grafos é um dos temas centrais em teoria da computação e ciência de dados, com aplicações que vão desde biologia computacional até análise de redes sociais. Em termos práticos, identificar subconjuntos altamente conectados de entidades pode revelar núcleos de influência, comunidades coesas ou grupos com maior potencial de propagação de informação. Este trabalho apresenta e implementa uma descrição

detalhada do problema do clique e propõe uma aplicação prática baseada em análise multicategoria para estimar suscetibilidade de engajamento viral em redes sociais.

A motivação central é a seguinte: em campanhas de marketing digital segmentadas, o objetivo não é simplesmente alcançar o maior número de pessoas, mas maximizar a **adesão** do público. O gatilho é comportamental: um usuário adere muito mais a um produto quando vários criadores que ele acompanha o endossam ao mesmo tempo — fenômeno conhecido como **prova social** [Cialdini 2006] e **efeito manada** (*bandwagon*) [Leibenstein 1950]. Para que esse reforço ocorra, os criadores precisam atingir a *mesma* audiência. O grupo ideal de campanha é, assim, o maior conjunto de criadores cujas audiências se sobrepõem par a par, que corresponde, formalmente, ao maior clique do grafo de audiências de cada categoria de interesse.

O trabalho é acompanhado de uma implementação funcional completa em TypeScript, capaz de gerar instâncias aleatórias reproduzíveis, construir os grafos por categoria, resolver o problema do clique máximo de forma exata e produzir um relatório HTML interativo com visualização dos grafos e comparativo entre categorias.

2. O Problema do Clique

Seja $G = (V, E)$ um grafo não direcionado, onde V é o conjunto de vértices e E o conjunto de arestas. Uma **clique** é um subconjunto $V' \subseteq V$ tal que todo par de vértices em V' está conectado por uma aresta em E . Em outras palavras, um clique é um subgrafo completo de G . O tamanho do clique é dado pelo número de vértices em V' .

O problema do clique pode ser formulado de duas formas principais:

- **Decisão:** dado um grafo G e um inteiro k , decidir se existe um clique de tamanho k em G .
- **Otimização:** encontrar o maior clique em G .

Ambas as versões são de grande interesse prático, mas a otimização é especialmente relevante para aplicações que buscam identificar o maior núcleo coeso possível. O problema do clique é conhecido por ser NP-completo [Karp 1972], o que significa que não se conhece algoritmo polinomial para resolvê-lo em geral.

2.1. Prova de NP-Completeness

Demonstramos formalmente que o problema de decisão associado ao clique é NP-completo [Karp 1972, Cormen et al. 2022]. Considere o problema de decisão

$$\text{CLIQUE} = \{ \langle G, k \rangle \mid G \text{ contém um clique de tamanho } k \}.$$

A demonstração segue as duas etapas canônicas: (i) mostrar que $\text{CLIQUE} \in \text{NP}$ e (ii) mostrar que CLIQUE é NP-difícil, por meio de uma redução polinomial a partir do 3-CNF-SAT, sabidamente NP-completo [Cook 1971].

CLIQUE $\in$ NP. Dado um certificado $V' \subseteq V$, verifica-se em tempo polinomial que $|V'| = k$ e que todo par de vértices $u, v \in V'$ satisfaz $(u, v) \in E$. Essa verificação examina $\binom{|V'|}{2} = O(|V|^2)$ pares, sendo portanto polinomial no tamanho da entrada. Se todos os pares estão conectados, V' é de fato um clique. Logo, $\text{CLIQUE} \in \text{NP}$.

Redução 3-CNF-SAT $\leq_p$ CLIQUE. Seja $\phi = C_1 \wedge C_2 \wedge \dots \wedge C_k$ uma fórmula booleana em 3-CNF, na qual cada cláusula $C_r = (l_1^r \vee l_2^r \vee l_3^r)$ contém exatamente três

literais (um literal é uma variável x_i ou sua negação $\neg x_i$). Construimos um grafo $G = (V, E)$ da seguinte forma:

- para cada cláusula C_r , criam-se três vértices v_1^r, v_2^r, v_3^r , um por literal, totalizando $3k$ vértices;
- adiciona-se a aresta (v_i^r, v_j^s) se e somente se $r \neq s$ (os vértices pertencem a cláusulas distintas) e os literais l_i^r e l_j^s não são complementares (não são x e $\neg x$ de uma mesma variável).

Observe que não há arestas entre vértices de uma mesma cláusula. A construção percorre todos os pares de vértices, custando $O((3k)^2)$ — polinomial em $|\phi|$. Afirmamos que ϕ é satisfatível se e somente se G possui um clique de tamanho k .

($\Rightarrow$) Suponha ϕ satisfatível. Em uma atribuição que a satisfaz, cada cláusula C_r possui ao menos um literal verdadeiro; selecione, para cada cláusula, um vértice v^r correspondente a um desses literais verdadeiros. O conjunto $V' = \{v^1, \dots, v^k\}$ possui exatamente k vértices. Para qualquer par v^r, v^s com $r \neq s$, ambos correspondem a literais verdadeiros e, por isso, não podem ser complementares (uma variável e sua negação não são simultaneamente verdadeiras). Como são de cláusulas distintas e consistentes, a aresta (v^r, v^s) pertence a E . Portanto, V' é um clique de tamanho k .

($\Leftarrow$) Suponha que G possua um clique V' de tamanho k . Como não existem arestas intra-cláusula, dois vértices de uma mesma cláusula nunca coexistem em V' ; logo, V' contém exatamente um vértice de cada uma das k cláusulas. Atribua o valor verdadeiro a cada literal correspondente a um vértice de V' . A atribuição é consistente: se dois vértices de V' representassem literais complementares, não haveria aresta entre eles, contradizendo o fato de V' ser um clique. Assim, cada cláusula possui ao menos um literal verdadeiro e ϕ é satisfatível.

Como a equivalência vale nas duas direções e a redução é computável em tempo polinomial, conclui-se que CLIQUE é NP-difícil. Somado a $\text{CLIQUE} \in \text{NP}$, segue que **CLIQUE é NP-completo**.

A NP-completude implica que não se conhece algoritmo polinomial para o caso geral; mais ainda, o problema é difícil até mesmo de aproximar dentro de qualquer fator constante [Feige et al. 1991]. Esse resultado motiva a adoção de um algoritmo de força bruta exato para instâncias de pequeno porte e de uma heurística de tempo polinomial (Seção 6.4) para cenários de maior escala.

3. Aplicação Prática: Engajamento Viral em Rede Social

Propomos uma aplicação do problema do clique para a seleção de grupos de influenciadores em campanhas baseadas em prova social. O objetivo é, dado um conjunto de usuários e múltiplas categorias de interesse (ex.: gosta de animais, esportes, tecnologia), identificar para cada categoria o maior grupo de criadores totalmente conectados (maior clique) — o conjunto de influenciadores que, endossando em conjunto o mesmo produto, maximiza a adesão do público da categoria.

A intuição parte de uma premissa comportamental bem estabelecida. Os candidatos a veicular a campanha são criadores de conteúdo — influenciadores, produtores de nicho — capazes de mobilizar suas audiências. A adesão de um usuário a um produto, porém, não depende do alcance isolado de um criador: ela é fortemente amplificada

quando **múltiplos criadores que o usuário acompanha endossam o mesmo produto simultaneamente**. Esse reforço positivo vindo do próprio meio social ativa a **prova social** [Cialdini 2006], o **efeito manada** [Leibenstein 1950] e a pressão de conformidade ao grupo [Asch 1951]; a exposição repetida, por fontes distintas e confiáveis, ainda reforça a afinidade com a marca [Zajonc 1968]. O usuário passa a interpretar o produto como um consenso do seu círculo social, e não como um anúncio isolado.

Para que esse reforço aconteça, os endossos precisam recair sobre *a mesma pessoa*. O escore de interação $s_{uv} \in [0, 1]$ abstrai justamente o grau de **sobreposição de audiência** entre dois criadores: quanto maior s_{uv} , maior a probabilidade de um mesmo seguidor acompanhar tanto u quanto v e, portanto, receber o endosso de ambos. Uma aresta $(u, v) \in E_a$ indica que as audiências de u e v se sobrepõem o suficiente para que o reforço seja relevante. Um grupo de criadores que forma um clique nesse grafo possui, por definição, *todas* as audiências sobrepostas par a par: o público comum ao grupo recebe o produto endossado por todos os seus membros ao mesmo tempo, sem elo fraco. O alcance agregado R_a estima o total de pessoas afetadas pelos criadores do grupo.

Da coesão à adesão. O tamanho do clique tem interpretação direta em termos de negócio, que formalizamos com um modelo probabilístico simples derivado de primeiros princípios. Num clique C_a^* de tamanho $k = |C_a^*|$, o público compartilhado recebe k endossos simultâneos, um por membro do grupo. Tratamos cada endosso como uma tentativa *independente* de conversão que, isoladamente, leva o usuário a aderir com probabilidade $q \in (0, 1)$ e, portanto, *não* o converte com probabilidade $1 - q$. Se os k endossos são independentes, a probabilidade de *todos* falharem é $(1 - q)^k$, e o usuário adere sempre que *ao menos um* deles converte. Pela regra do complemento, chega-se à **curva de adesão**

$$p(\text{adesão}) = 1 - (1 - q)^k.$$

A fórmula não foi ajustada a dados empíricos. Ela é a mesma expressão do modelo *noisy-OR* em redes probabilísticas e da confiabilidade de k fontes redundantes ligadas em paralelo, e a adotamos por traduzir de modo mínimo a intuição de prova social — mais endossos independentes elevam a adesão, com saturação natural rumo a 1. O parâmetro q é constante global do modelo (valor ilustrativo $q = 0,15$), e a hipótese de independência entre endossos é uma simplificação deliberada, já que audiências reais se correlacionam e tendem a reduzir a adesão observada. A função é estritamente crescente em k , ainda que com retornos marginais decrescentes: para $q = 0,15$, obtém-se $p(1) = 0,15$, $p(3) \approx 0,39$ e $p(4) \approx 0,48$. Como a adesão esperada cresce monotonicamente com k , o *maior* clique é sempre o de maior potencial de adesão — o que fundamenta a escolha do problema do clique **máximo** como objetivo, e não de um clique qualquer.

4. Modelagem Formal da Aplicação

Seja U o conjunto de usuários e A o conjunto de categorias. Cada usuário $u \in U$ possui um vetor de preferências binárias $\text{pref}(u, a) \in \{0, 1\}$ para cada $a \in A$, e um alcance $\text{reach}(u) \in \mathbb{Z}^+$ que representa o número de seguidores.

Para cada categoria $a \in A$, define-se o grafo $G_a = (V_a, E_a)$, onde:

- $V_a = \{u \in U \mid \text{pref}(u, a) = 1\}$ é o subconjunto de usuários com interesse na categoria a ;

- $(u, v) \in E_a$ se e somente se o escore de interação $s_{uv} \geq \tau$, sendo τ um limiar definido.

O objetivo é encontrar, para cada $a \in A$, o maior clique $C_a^* \subseteq V_a$. O alcance agregado do clique é definido como:

$$R_a = \sum_{u \in C_a^*} \text{reach}(u)$$

O ranking das categorias é determinado primeiramente por $|C_a^*|$ (tamanho do clique) em ordem decrescente e, em caso de empate, por R_a (alcance agregado) também em ordem decrescente. O tamanho do clique tem leitura comportamental direta: corresponde ao número de endossos simultâneos sobre o público compartilhado, que rege a adesão esperada pela curva $p = 1 - (1 - q)^k$ apresentada na seção anterior.

5. Metodologia do Estudo

A metodologia adotada segue as etapas abaixo:

1. **Geração da instância:** criar aleatoriamente um conjunto de usuários U e respostas binárias para cada categoria $a \in A$, com escore de interação pairwise $s_{uv} \in [0, 1]$.
2. **Construção dos grafos:** para cada categoria a , filtrar os usuários elegíveis V_a e construir G_a adicionando arestas onde $s_{uv} \geq \tau$.
3. **Solução do clique máximo:** aplicar o algoritmo de busca exata em cada G_a , obtendo C_a^* .
4. **Ranking:** ordenar as categorias por $|C_a^*|$ e, como desempate, por R_a .
5. **Relatório:** gerar relatório HTML interativo com visualizações dos grafos e gráficos comparativos.
6. **Avaliação comparativa:** executar baseline e heurística sobre as mesmas instâncias, variando o tamanho de entrada n , e medir tempo e qualidade da solução (Seção 7).

Os parâmetros padrão utilizados no estudo de caso são: $|U| = 30$ usuários, $|A| = 5$ categorias (*animals, sports, technology, music, food*), probabilidade de preferência $p = 0,5$, limiar de interação $\tau = 0,6$ e semente determinística 42. Todos esses parâmetros são centralizados no arquivo `config.ts`, que serve como ponto único de configuração da simulação: alterar seus valores e executar novamente o sistema é suficiente para explorar diferentes cenários sem modificar a lógica de análise.

5.1. Ambiente de Execução

Os experimentos foram executados em um processador Intel Core i5-10400 (6 núcleos físicos, 12 *threads*, 2,90 GHz), com 64 GB de RAM, sob o sistema operacional Windows 11 (*build* 10.0.26200). O código executa sobre Node.js 22.22.2, escrito em TypeScript 5.9.3 e interpretado via `tsx`; a suíte de testes usa o *framework* `vitest` 4.1.5. Os tempos de execução do solver foram medidos com `performance.now()`, em milissegundos, com uma execução por par $(n, \text{semente})$. Como todo o pipeline é determinístico (PRNG semeado), os resultados são integralmente reproduzíveis.

6. Implementação

A implementação foi desenvolvida em TypeScript, organizada em módulos com responsabilidades bem definidas: `models.ts` (interfaces e tipos), `generator.ts` (geração de instâncias), `graph.ts` (estrutura de dados do grafo), `solver.ts` (algoritmo de clique máximo), `analyzer.ts` (orquestração da análise) e `report.ts` (geração do relatório HTML).

6.1. Geração de Instâncias

A classe `InstanceGenerator` utiliza o algoritmo Mulberry32 como gerador pseudo-aleatório (PRNG) determinístico. Dado um valor de semente, o gerador produz a mesma sequência de números a cada execução, garantindo que os experimentos sejam reproduzíveis. Para cada par de usuários (u, v) , é gerado um escore de interação $s_{uv} \in [0, 1]$, armazenado com chave canônica $\min(u, v), \max(u, v)$ para garantir acesso simétrico eficiente.

6.2. Representação do Grafo

A classe `Graph` representa o grafo por lista de adjacências utilizando `Map<number, Set<number>>`, estrutura que oferece operações de inserção e consulta de arestas em tempo $O(1)$ amortizado. As arestas são armazenadas simetricamente (tanto $u \rightarrow v$ quanto $v \rightarrow u$), e o contador de arestas é mantido explicitamente para evitar recomputação. A construção de G_a consiste em filtrar os usuários por preferência e percorrer todos os pares elegíveis, adicionando a aresta (u, v) sempre que $s_{uv} \geq \tau$.

6.3. Algoritmo de Clique Máximo

A classe `BruteSolver` implementa um algoritmo exato de força bruta com terminação antecipada. O algoritmo enumera subconjuntos de vértices em ordem decrescente de tamanho — de $|V_a|$ até 1 — e retorna o primeiro subconjunto que constitui um clique. Como a enumeração inicia pelo maior tamanho possível, o primeiro clique encontrado é necessariamente o máximo.

A enumeração de combinações é implementada como um gerador *lazy* por retrocesso recursivo (*backtracking*), o que evita a alocação antecipada de todos os subconjuntos em memória. A verificação de clique testa todos os pares do subconjunto em $O(k^2)$, onde k é o tamanho do subconjunto. A complexidade global do algoritmo é $O(2^n \cdot n^2)$, sendo adequado para grafos com $|V_a| \leq 25$ vértices, conforme o perfil das instâncias geradas neste trabalho.

Para comparação, o algoritmo de Bron-Kerbosch [Bron and Kerbosch 1973] oferece uma alternativa exata de retrocesso com melhor desempenho em média ao podar ramos não promissores via pivoteamento; o algoritmo de força bruta foi adotado como baseline por sua clareza conceitual e adequação ao tamanho das instâncias de estudo.

6.4. Heurística Gulosa por Grau

Como contraponto ao baseline exato, projetamos e implementamos uma heurística gulosa (classe `GreedySolver`) que resolve o problema do clique em tempo polinomial, abrindo mão da garantia de otimalidade em troca de escalabilidade.

Estratégia. O algoritmo ordena os vértices de G_a por grau decrescente e os percorre uma única vez, mantendo um clique parcial C , inicialmente vazio. Cada vértice v é incorporado a C se e somente se for adjacente a *todos* os vértices já presentes em C . A intuição é que vértices de grau elevado têm maior probabilidade de pertencer a um clique grande, por disporem de mais vizinhos com quem formar conexões mútuas. Empates de grau são resolvidos pela ordem crescente de identificador do vértice, o que torna a saída determinística e reproduzível — coerente com a geração de instâncias por semente fixa adotada no restante do sistema.

Complexidade. O cálculo dos graus custa $O(n)$, a ordenação $O(n \log n)$ e a verificação executa n iterações, cada uma verificando a adjacência do vértice corrente contra o clique parcial em $O(k)$, com $k \leq n$. No pior caso, a complexidade de tempo é $O(n^2)$ e o espaço é $O(n)$, onde $n = |V_a|$. Isso contrasta radicalmente com o $O(2^n \cdot n^2)$ do baseline exato, permitindo aplicar a heurística a grafos ordens de magnitude maiores.

Características. Por construir uma única solução de forma determinística e sem retrocesso, a heurística é rápida, mas *não garante o ótimo global*: pode ficar presa em um clique maximal (não extensível) de tamanho inferior ao clique máximo. Isso ocorre, por exemplo, quando um vértice de grau elevado, escolhido logo no início, é adjacente a poucos vértices que, por sua vez, não formam um clique entre si — bloqueando o acesso a um clique maior situado em outra região do grafo. A magnitude dessa perda de qualidade e o ganho de desempenho frente ao baseline são avaliados experimentalmente na Seção 7, por meio de testes em diferentes tamanhos de entrada.

Ambos os algoritmos — baseline e heurística — expõem a mesma interface (`CliqueAlgorithm`), de modo que podem ser injetados de forma intercambiável no `ViralAnalyzer` e no arcabouço de experimentos. Isso garante que a comparação entre eles seja realizada sobre exatamente as mesmas instâncias, isolando as diferenças de tempo e de qualidade da solução como efeito exclusivo do algoritmo.

6.5. Análise e Ranking

A classe `ViralAnalyzer` orquestra o fluxo completo: para cada categoria $a \in A$, constrói G_a , executa o `BruteSolver` e calcula o alcance agregado $R_a = \sum_{u \in C_a^*} \text{reach}(u)$. Ao final, as categorias são ordenadas por $|C_a^*|$ decrescente e R_a como critério de desempate, produzindo o ranking de potencial viral.

6.6. Relatório Visual

A classe `ReportGenerator` produz um arquivo HTML auto-contido com três tipos de visualização: (i) gráficos de barras comparando $|C_a^*|$ e R_a entre categorias, gerados com `Chart.js`; (ii) uma tabela de todos os usuários, destacando os membros de algum clique máximo; e (iii) uma visualização interativa do grafo G_a para cada categoria, gerada com `vis.js`, em que os nós do clique são destacados em laranja e os demais em azul.

Para evitar defeitos de renderização em contêineres inicialmente ocultos, a inicialização dos grafos `vis.js` é realizada de forma *lazy*: a função de construção de cada grafo é armazenada em um dicionário e executada somente na primeira vez que o usuário navega até a aba da categoria correspondente.

7. Resultados

Executando o sistema com os parâmetros padrão — semente 42, 30 usuários, 5 categorias, $\tau = 0,6$ — obtêm-se os resultados apresentados na Tabela 1.

Tabela 1. Resultados por categoria (instância padrão: 30 usuários, $\tau = 0,6$, semente 42)

Categoria	$ V_a $	$ E_a $	$ C_a^* $	R_a (seguidores)
animals	18	65	4	1.201.515
technology	15	36	4	851.826
food	15	40	4	842.877
sports	20	76	4	538.537
music	9	15	3	655.851

Quatro das cinco categorias apresentaram clique máximo de tamanho 4; a categoria *music*, com menor grafo ($|V_a| = 9$), atingiu clique de tamanho 3 e ficou em último lugar no ranking de potencial viral, apesar de possuir alcance agregado superior ao de *sports*. Isso ilustra o comportamento do critério de desempate: o tamanho do clique é o fator primário de ranqueamento, e o alcance agregado apenas diferencia categorias empatadas.

A categoria *animals* liderou o ranking com clique de tamanho 4 e alcance agregado de 1.201.515 seguidores, sendo a recomendação do sistema como melhor grupo para uma campanha de reforço. Os quatro membros do clique — User 1 (300.950 seguidores), User 5 (134.211), User 10 (463.992) e User 22 (302.362) — formam um subgrafo completamente conectado dentro do subgrafo de interessados em *animals*: o público que acompanha esses criadores recebe quatro endossos simultâneos do produto. Pela curva de adesão com $q = 0,15$, isso corresponde a uma adesão esperada de $1 - (1 - 0,15)^4 \approx 48\%$, contra apenas 15% de um único influenciador — quantificando o ganho do reforço em grupo.

O relatório HTML gerado permite inspecionar visualmente cada grafo G_a , navegar entre categorias e verificar quais usuários compõem o clique máximo de cada segmento, facilitando a interpretação e comunicação dos resultados para equipes de marketing.

7.1. Avaliação Comparativa: Baseline × Heurística

Para avaliar o compromisso entre exatidão e desempenho, executamos o baseline exato (BruteSolver) e a heurística gulosa (GreedySolver) sobre as mesmas instâncias, variando $n = |V_a|$ de 6 a 20 (categoria única com prefProb = 1, 10 sementes por n , $\tau = 0,6$; força bruta limitada a $n \leq 20$ por viabilidade). A Tabela 2 resume os resultados.

Desempenho e escala. O tempo do baseline cresce explosivamente — de 0,09 ms em $n = 6$ a 879 ms em $n = 20$, quase quatro ordens de magnitude —, coerente com $O(2^n \cdot n^2)$; a heurística $O(n^2)$ mantém-se em $\sim 0,02$ ms, e o *speedup* salta de $3\times$ a $\sim 45.751\times$, crescendo indefinidamente além do teto. (Tempos de n pequeno sofrem ruído de aquecimento do JIT, por serem medições únicas.)

Tabela 2. Baseline (força bruta) × heurística (guloso) por tamanho de entrada. Médias sobre 10 sementes, $\tau = 0,6$. Ambiente na Seção 5.1.

n	t_{bf} (ms)	t_{heur} (ms)	$speedup$	$ C _{bf}$	$ C _{heur}$	% ótimo
6	0,092	0,029	$3\times$	2,60	2,50	90 %
8	0,374	0,007	$56\times$	3,00	2,70	70 %
10	0,635	0,008	$80\times$	3,60	3,10	50 %
12	2,405	0,013	$181\times$	3,90	3,70	80 %
14	9,865	0,015	$637\times$	4,30	4,10	80 %
16	44,668	0,016	$2.861\times$	4,50	4,00	50 %
18	202,926	0,018	$11.083\times$	4,30	3,90	60 %
20	878,880	0,019	$45.751\times$	4,60	4,30	70 %

Qualidade e compromisso. O guloso atingiu o ótimo em 55 das 80 execuções (68,8%); quando erra, a razão de qualidade $|C|_{heur}/|C|_{bf}$ permanece entre 86% e 96%. A taxa de acerto oscila (50%–90%) sem tendência clara em n , conforme a topologia aleatória favoreça ou não a escolha por grau. Nos 25 casos subótimos o guloso devolveu clique menor *porém* muito mais rápido: para $|V_a| \leq 20$ a força bruta compensa (garante o ótimo a custo tolerável), mas à medida que n cresce ela se torna proibitiva e a heurística passa a ser a única opção viável, com perda de qualidade tipicamente $< 15\%$. A dispersão $speedup \times$ qualidade no relatório HTML destaca exatamente esses casos subótimos-porém-rápidos.

8. Conclusão

Este trabalho apresentou o problema do clique em grafos, demonstrou sua NP-completude e propôs e implementou uma aplicação prática para seleção de grupos de influenciadores em campanhas baseadas em prova social em redes sociais.

A implementação completa em TypeScript compreende: geração determinística e reproduzível de instâncias, representação eficiente do grafo por lista de adjacências, **dois** algoritmos de clique máximo intercambiáveis pela interface `CliqueAlgorithm` — o baseline exato de força bruta com terminação antecipada ($O(2^n \cdot n^2)$) e a heurística gulosa por grau ($O(n^2)$) —, análise multicategoria com ranking por tamanho de clique e alcance agregado, um arcabouço de *benchmark* automatizado por tamanho de entrada e a geração de relatório HTML interativo.

Os resultados esperados — a força bruta garantindo o ótimo a custo exponencial e o guloso trocando otimalidade por escala — confirmaram-se experimentalmente. Na instância padrão, o sistema apontou a categoria *animals* como o melhor núcleo de reforço (clique de tamanho 4, adesão estimada de 48%). Na avaliação comparativa (Seção 7.1), a heurística mostrou-se de dezenas a mais de quarenta mil vezes mais rápida que o baseline, atingindo o ótimo em 69% das execuções e, quando falha, entregando cliques com qualidade média superior a 86%. Conclui-se que a força bruta é preferível nas instâncias de pequeno porte, por garantir o ótimo a custo ainda tolerável; à medida que o grafo cresce, a busca exata torna-se proibitiva ($O(2^n \cdot n^2)$) e a heurística passa a ser a opção viável, aceitando-se uma perda de qualidade tipicamente inferior a 15%.

Como trabalhos futuros, aponta-se: (i) algoritmos exatos mais eficientes que a

força bruta pura, como Bron-Kerbosch com pivoteamento [Bron and Kerbosch 1973], e metaheurísticas (por exemplo, *simulated annealing*) como alternativas à heurística gulosa; (ii) a incorporação de grafos ponderados, em que o escore de interação s_{uv} influencie diretamente o ranking em vez de apenas determinar a existência da aresta; (iii) o refinamento da curva de adesão, com q variável por categoria/nicho e endossos correlacionados em vez de independentes; e (iv) a avaliação em grafos reais provenientes de plataformas de redes sociais. O problema do clique oferece, assim, uma lente rigorosa e matematicamente fundamentada para identificar os grupos com maior potencial de reforço e adesão em qualquer segmento de interesse.

Referências

- Asch, S. E. (1951). Effects of group pressure upon the modification and distortion of judgments. In Guetzkow, H., editor, *Groups, Leadership and Men*, pages 177–190. Carnegie Press.
- Bron, C. and Kerbosch, J. (1973). Algorithm 457: Finding all cliques of an undirected graph. *Communications of the ACM*, 16(9):575–577.
- Cialdini, R. B. (2006). *Influence: The Psychology of Persuasion*. Harper Business, revised edition.
- Cook, S. A. (1971). The complexity of theorem-proving procedures. In *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing (STOC)*, pages 151–158.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*. MIT Press, 4th edition. Seção 34.5.1, Teorema 34.11, pp. 1081–1084.
- Feige, U., Goldwasser, S., Lovász, L., Safra, S., and Szegedy, M. (1991). Approximating clique is almost NP-complete. In *Proceedings of the 32nd Annual Symposium on Foundations of Computer Science*, pages 2–12.
- Karp, R. M. (1972). Reducibility among combinatorial problems. In Miller, R. E. and Thatcher, J. W., editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press.
- Leibenstein, H. (1950). Bandwagon, snob, and veblen effects in the theory of consumers' demand. *The Quarterly Journal of Economics*, 64(2):183–207.
- Zajonc, R. B. (1968). Attitudinal effects of mere exposure. *Journal of Personality and Social Psychology*, 9(2, Pt.2):1–27.